Digital Image Processing
Spring 2025
Lab:9 Image Restoration (denoising)

Name	
Student ID	
Date	

Objectives:

1. Learn how to add different types of noise to an image.
2. Understand and implement various filtering techniques.
3. Analyze the performance of filters on specific noise types.
4. Develop problem-solving skills through challenging tasks.

1. Adding Noise to Images

In this section, you will write and analyze functions to add different types of noise to images. Each function will be explained with detailed comments.

1.1. Gaussian Noise

Gaussian noise is a statistical noise with a probability density function equal to that of the normal distribution (also called Gaussian distribution).

```
def add_gaussian_noise(image, mean=0, std=1):
    """
    Adds Gaussian noise to an image.

    Parameters:
    - image: Input image to which noise will be
added.
    - mean: Mean of the Gaussian distribution.
Default is 0.
    - std: Standard deviation of the Gaussian
distribution. Default is 1.

    Returns:
    - noisy_image: Image with added Gaussian noise.
```

```
    """
    # Generate Gaussian noise based on the shape of
the image
    gaussian_noise = np.random.normal(mean, std,
image.shape)

    # Add the noise to the original image and clip
the result to stay within valid range
    noisy_image = image + gaussian_noise
    noisy_image = np.clip(noisy_image, 0, 255)

    return noisy_image
```

1.2. Rayleigh Noise

Rayleigh noise follows a Rayleigh probability distribution. This noise is most common in the case of radar systems.

```
def add_rayleigh_noise(image, scale=1):
    """
    Adds Rayleigh noise to an image.

    Parameters:
    - image: Input image to which noise will be
added.
    - scale: Scale parameter for the Rayleigh
distribution.

    Returns:
    - noisy_image: Image with added Rayleigh noise.
    """
    # Generate Rayleigh noise based on the shape of
the image
    rayleigh_noise = np.random.rayleigh(scale,
image.shape)
```

```
    # Add the noise to the original image and clip
the result to stay within valid range
    noisy_image = image + rayleigh_noise
    noisy_image = np.clip(noisy_image, 0, 255)

    return noisy_image
```

1.3. Exponential Noise

Exponential noise follows an exponential distribution. This type of noise is typically asymmetric and more concentrated in lower values.

```
def add_exponential_noise(image, scale=1):
    """
    Adds Exponential noise to an image.

    Parameters:
    - image: Input image to which noise will be
added.
    - scale: Scale parameter for the Exponential
distribution.

    Returns:
    - noisy_image: Image with added Exponential
noise.
    """
    # Generate Exponential noise based on the shape
of the image
    exponential_noise =
np.random.exponential(scale, image.shape)

    # Add the noise to the original image and clip
the result to stay within valid range
    noisy_image = image + exponential_noise
```

```
    noisy_image = np.clip(noisy_image, 0, 255)

    return noisy_image
```

1.4. Salt & Pepper Noise

Salt & Pepper noise randomly replaces some of the pixels in an image with either minimum or maximum intensity values (black or white pixels).

```
def add_salt_pepper_noise(image, salt_prob=0.05,
pepper_prob=0.05):
    """
    Adds Salt & Pepper noise to an image.

    Parameters:
    - image: Input image to which noise will be
added.
    - salt_prob: Probability of adding salt (white)
noise.
    - pepper_prob: Probability of adding pepper
(black) noise.

    Returns:
    - noisy_image: Image with added Salt & Pepper
noise.
    """
    noisy_image = image.copy()

    # Salt noise (white pixels)
    num_salt = np.ceil(salt_prob * image.size)
    coords = [np.random.randint(0, i - 1,
int(num_salt)) for i in image.shape]
    noisy_image[coords[0], coords[1]] = 255

    # Pepper noise (black pixels)
```

```
    num_pepper = np.ceil(pepper_prob * image.size)
    coords = [np.random.randint(0, i - 1,
int(num_pepper)) for i in image.shape]
    noisy_image[coords[0], coords[1]] = 0

    return noisy_image
```

1.5. Uniform Noise

Uniform noise is added using a uniform distribution, where noise values are uniformly distributed over a specified range.

```
def add_uniform_noise(image, low=0, high=10):
    """
    Adds Uniform noise to an image.

    Parameters:
    - image: Input image to which noise will be
added.
    - low: Lower bound of the uniform noise.
    - high: Upper bound of the uniform noise.

    Returns:
    - noisy_image: Image with added Uniform noise.
    """
    # Generate Uniform noise based on the shape of
the image
    uniform_noise = np.random.uniform(low, high,
image.shape)

    # Add the noise to the original image and clip
the result to stay within valid range
    noisy_image = image + uniform_noise
    noisy_image = np.clip(noisy_image, 0, 255)
```

```
    return noisy_image
```

2. Adding Noise to the image

An example of how Gaussian noise can be added to the image is provided in the script below

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load an example image
image = cv2.imread('example_image.jpg',
cv2.IMREAD_GRAYSCALE)

# Add Gaussian noise to the image
noisy_image = add_gaussian_noise(image, mean=0,
std=20)

# Display the noisy image
plt.imshow(noisy_image, cmap='gray')
plt.title("Image with Gaussian Noise")
plt.show()
```

Task 1:

1. Apply Gaussian noise of different mean and variance to the same input image, observe the histogram and the differences. (at least five combinations are required)
2. Apply salt and pepper noise of different probabilities for salt and pepper, observe the image, its histogram and the differences. (at least five combinations are required)
3. Apply uniform noise of different low and high values, observe the image, its histogram and the differences. (at least five combinations are required)
4. Apply the remaining noise types to the input image and observe the same.

2. Implementing filters

I have implemented box filter (mean filter) with the help of loops. The source code is provided below

```
def apply_box(image_input, box, filt_size):
    # Calculate the padding size based on the
filter size
    pad_size = int(np.floor(filt_size / 2))

    # Pad the input image symmetrically to handle
border pixels (mirror padding)
    image_padded = np.pad(image_input,
                          pad_width=((pad_size,
pad_size), (pad_size, pad_size)),
                          mode='symmetric')

    # Create a deep copy of the input image to
store the filtered result
    image_box = copy.deepcopy(image_input)

    # Get the dimensions (number of rows and
columns) of the original input image
    row, column = image_input.shape

    # Loop over each pixel in the original image
    for i in range(row):
        for j in range(column):
            # Extract the current patch of the
padded image based on the filter size
            patch_curr = image_padded[i:i +
```

```
filt_size, j:j + filt_size]

            # Multiply the current patch with the
box filter element-wise
            results_box = box * patch_curr

            # Assign the sum of the multiplied
patch as the new pixel value
            image_box[i, j] = np.sum(results_box)

    # Return the filtered image
    return image_box
```

You are required to modify the code above to implement the functions for the following filters

1. **Geometric Mean Filter**
2. **Harmonic Mean Filter**
3. **Contraharmonic Mean Filter**

Order Statistic Filters

1. **Median Filter**
2. **Max Filter**
3. **Min Filter**
4. **Midpoint Filter**
5. **Alpha-Trimmed Mean Filter**

Task 2:

1. Apply Gaussian noise on the input image, apply median filter of any window size to see if it improves the results
2. Apply Gaussian noise of 0 mean and 0.0009 variance. Try denoising the image using
    1. 3x3 and 5x5 mean filter
    2. 3x3 and 5x5 geometric mean filter
3. Apply Gaussian noise of 0 mean and 0.09 variance. Try denoising the image using the filters discussed in 2nd task. If image doesn't improve, try increasing the window size to 7x7 or higher to see if there is any progress.

4. Apply salt & pepper noise with 0.1 probability for both salt and pepper. Apply 3x3 median filter to see if it improves.
5. Apply salt & pepper noise with 0.2 probability for both salt and pepper. Apply 3x3 median filter to see if it improves. If it doesn't improve, apply the following
    1. 5x5 median filter
    2. 7x7 median filter
    3. 3x3 median filter with 4 iterations
6. Add Gaussian noise of 0 mean and 0.0009 variance followed by addition of salt & pepper noise with probability of 0.1 for salt and probability of 0 for pepper. Apply harmonic mean filter of size 5x5 and observe the results. If results aren't improved, try increasing the window size.
7. Add salt & pepper noise of probability of 0.2 for salt and probability of 0 for pepper. Apply harmonic mean filter of different window sizes and mention the window size that works well.
8. Apply alpha trimmed mean filter to the image with 0 mean and 0.009 variance for the Gaussian noise followed by application of salt & pepper noise with salt probability of 0.1 and pepper probability of 0.1. Use window size of 7x7 and use different values of d such as 4, 6, 8, 10, 12, 14, 16 and mention the one that works well.
9. Add salt & pepper noise of probability of 0.2 for salt and probability of 0 for pepper. Apply min filter with window size of 3
10. Add salt & pepper noise of probability of 0.5 for salt and probability of 0 for pepper. Apply min filter with window size of 3. If it doesn't work, use other window sizes and choose the best one.
11. Add salt & pepper noise of probability of 0.2 for pepper and probability of 0 for salt. Apply max filter with window size of 3
12. Add salt & pepper noise of probability of 0.5 for pepper and probability of 0 for salt. Apply max filter with window size of 3. If it doesn't work, use other window sizes and choose the best one.

Task 3:

1. Apply salt & pepper noise with 0.1 probability for both salt and pepper. Apply the following to see if they are of any benefit
    1. 5x5 min filter
    2. 5x5 max filter
    3. 5x5 harmonic mean filter
    4. 5x5 geometric mean filter
2. Add salt & pepper noise of probability of 0.2 for salt and probability of 0 for pepper. Apply the following to see if they are of any use
    1. 5x5 max filter
    2. Contraharmonic mean filter of size 5x5 and Q=2

3. Add salt & pepper noise of probability of 0.2 for pepper and probability of 0 for salt. Apply the following to see if they are of any use
    1. 5x5 min filter
    2. Contraharmonic mean filter of size 5x5 and Q= -2
    3. 5x5 harmonic mean filter

Submission

Submit the following:

- Screenshots of the images before and after applying each filter.
- The code for each task.
- A brief report discussing how each filter affects image sharpness, blurring, and enhancement.